

ui.slider 全面详解（基于 NiceGUI 文档）

ui.slider 是 NiceGUI 中用于实现数值范围选择的核心交互组件，基于 Quasar 的 QSlider 组件实现。其核心价值在于通过滑动操作直观选择连续或离散数值，支持自定义数值范围、步长、事件节流和样式定制，适用于音量调节、亮度控制、参数配置等需要精准或粗略数值选择的场景。以下从核心特性、基础用法、高级功能等维度展开全面解析。

一、核心基础

1. 组件本质与核心特性

- 底层依赖：**基于 Quasar 的 QSlider 组件，继承其滑动交互逻辑、数值约束和样式体系。
- 核心功能：**支持设置数值上下界、步长，实时反馈当前数值，支持事件节流（避免高频触发回调），可禁用和样式定制。
- 交互特性：**拖动滑块时实时更新数值，释放滑块时触发 `on_change` 回调（默认行为），支持通过 `update:model-value` 事件监听实时变化。

2. 初始化参数（核心配置）

参数名	类型	说明	默认值
min	float / int	数值下界（最小值）	0
max	float / int	数值上界（最大值）	100
step	float / int	滑动步长（每次拖动的数值增量，0 表示连续数值）	1
value	float / int	初始数值（需在 min 和 max 之间）	中间值 ((min+max)/2)
on_change	Callable	滑块释放时触发的回调函数（ <code>e.value</code> 为当前数值）	-

二、基础使用示例

1. 最简用法（数值展示联动）

展示基础数值范围、初始值配置，以及与标签的实时联动：

```

from nicegui import ui

# 基础滑块（0-100，初始值50）
slider = ui.slider(min=0, max=100, value=50)
# 实时显示当前数值
ui.label().bind_text_from(slider, 'value', lambda x: f'当前值: {x}')

ui.run()

```

效果：滑块拖动时，下方标签实时更新显示当前数值，释放滑块时触发 `on_change` 回调（若配置）。

2. 禁用与启用控制

通过按钮控制滑块的禁用状态，禁用后滑块灰度显示且不可拖动：

```

from nicegui import ui

slider = ui.slider(min=0, max=100, value=50)

# 禁用/启用按钮
with ui.row().classes('mt-2 gap-2'):
    ui.button('禁用滑块', on_click=slider.disable)
    ui.button('启用滑块', on_click=slider.enable)

ui.run()

```

效果：点击“禁用滑块”后，滑块不可拖动且视觉灰度；点击“启用滑块”后恢复正常交互。

三、核心功能与进阶用法

1. 事件节流与实时监听

通过 `throttle` 控制事件触发频率，支持“领先事件”“尾随事件”配置，适配不同交互需求：

```

from nicegui import ui

# 1. 默认行为（领先+尾随事件，节流1秒）
ui.label('默认（领先+尾随事件）')
ui.slider(min=0, max=10, step=0.1, value=5).props('label-always') \
    .on('update:model-value', lambda e: ui.notify(f'数值: {e.args[0]}'), throttle=1.0)

# 2. 仅领先事件（拖动开始时立即触发，之后节流期间不重复触发）
ui.label('仅领先事件').classes('mt-4')
ui.slider(min=0, max=10, step=0.1, value=5).props('label-always') \
    .on('update:model-value', lambda e: ui.notify(f'数值: {e.args[0]}'), throttle=1.0,
    trailing_events=False)

# 3. 仅尾随事件（拖动结束后节流时间到触发）
ui.label('仅尾随事件').classes('mt-4')
ui.slider(min=0, max=10, step=0.1, value=5).props('label-always') \

```

```
.on('update:model-value', lambda e: ui.notify(f'数值: {e.args[0]}'), throttle=1.0,
leading_events=False)

ui.run()
```

效果:

- 默认模式: 拖动开始立即触发一次, 拖动中每 1 秒触发一次, 释放后再触发一次;
- 仅领先模式: 仅拖动开始时触发一次;
- 仅尾随模式: 拖动结束后 1 秒触发一次。

2. 自定义数值范围与步长

设置非默认范围和步长, 支持整数、小数数值选择:

```
from nicegui import ui

with ui.column().classes('gap-4'):
    # 整数范围 (1-10, 步长2)
    ui.label('整数选择 (1-10, 步长2)')
    int_slider = ui.slider(min=1, max=10, step=2, value=5)
    ui.label().bind_text_from(int_slider, 'value')

    # 小数范围 (0-1, 步长0.1)
    ui.label('小数选择 (0-1, 步长0.1)')
    float_slider = ui.slider(min=0, max=1, step=0.1, value=0.5)
    ui.label().bind_text_from(float_slider, 'value', lambda x: f'{x:.1f}')

ui.run()
```

效果:

- 整数滑块仅可选 1、3、5、7、9;
- 小数滑块支持 0.0、0.1、...、1.0 等数值, 标签格式化显示一位小数。

3. 数据绑定 (双向同步)

通过 `bind_value` 实现滑块与数据对象的双向绑定, 数据变化时组件自动更新, 反之亦然:

```
from nicegui import ui

class AppConfig:
    def __init__(self):
        self.volume = 70 # 初始音量

config = AppConfig()

# 滑块与数据对象绑定
slider = ui.slider(min=0, max=100, value=config.volume).bind_value(config, 'volume')

# 显示绑定数据的实时状态
```

```
ui.label().bind_text_from(config, 'volume', lambda x: f'音量: {x}%')

# 手动修改数据对象，组件自动同步
ui.button('静音 (0%)', on_click=lambda: setattr(config, 'volume', 0)).classes('mt-2')
ui.button('最大音量 (100%)', on_click=lambda: setattr(config, 'volume', 100)).classes('mt-2')

ui.run()
```

效果：

- 拖动滑块改变音量，`config.volume` 自动同步；
- 点击按钮修改 `config.volume`，滑块位置自动调整，标签实时更新。

4. 样式定制（颜色、标签、尺寸）

通过 `props` 和 `classes` 定制滑块颜色、标签显示、尺寸等样式：

```
from nicegui import ui

with ui.column().classes('gap-4'):
    # 自定义滑块颜色（绿色）
    ui.slider(min=0, max=100, value=50).props('color=green label-always')

    # 紧凑模式+大尺寸
    ui.slider(min=0, max=100, value=50).props('dense size=lg')

    # 隐藏刻度，仅显示滑块
    ui.slider(min=0, max=100, value=50).props('no-ticks')

    # 自定义宽度和滑块样式
    ui.slider(min=0, max=100, value=50).classes('w-64') \
        .style('--q-slider-thumb-color: #667eea; --q-slider-track-color: #e2e8f0;')

ui.run()
```

效果：

- 四个滑块分别展示不同样式，支持颜色、尺寸、刻度显示、滑块 / 轨道颜色的定制。

四、核心属性与方法

1. 常用属性

属性名	类型	说明
classes	Classes[Self]	CSS 类（支持 Tailwind/Quasar 样式）
enabled	BindableProperty	是否启用（可绑定数据动态控制，禁用后不可交互）
html_id	str	HTML 元素 ID（2.16.0+ 版本支持）
min	float / int	数值下界（可动态修改）

属性名	类型	说明
max	float / int	数值上界（可动态修改）
step	float / int	滑动步长（可动态修改）
value	BindableProperty	当前数值（可绑定数据，需在 min 和 max 之间）
visible	BindableProperty	是否可见（可绑定数据）
props	Props[Self]	Quasar 特性属性（如 <code>color</code> 颜色、 <code>label-always</code> 始终显示标签）

2. 关键方法

(1) 状态控制

- `disable()`：禁用组件（不可拖动，视觉灰度）
- `enable()`：启用组件
- `set_enabled(value: bool)`：设置启用状态（True/False）
- `set_visibility(visible: bool)`：设置组件可见性

(2) 属性修改

- `set_value(value: float / int)`：动态修改当前数值（需在 min 和 max 之间，触发 `on_change` 回调）
- `set_min(min: float / int)`：动态修改数值下界
- `set_max(max: float / int)`：动态修改数值上界
- `set_step(step: float / int)`：动态修改滑动步长
- `update()`：修改属性后，调用此方法刷新界面（绑定数据时无需手动调用）

(3) 事件与绑定

- `on_change(callback)`：绑定滑块释放时的事件（`e.value` 为当前数值）
- `on(type: str, handler)`：订阅任意 DOM 事件（如 `update:model-value` 监听实时变化）
- `bind_value(target_object, target_name)`：双向绑定当前数值到目标对象的属性
- `bind_min_from(target_object, target_name)`：单向绑定数值下界从目标对象

(4) 其他实用方法

- `tooltip(text: str)`：为组件添加悬停提示
- `delete()`：彻底删除组件
- `mark(*markers)`：添加标记（用于测试或元素查询）

五、使用场景与注意事项

1. 适用场景

- 参数配置：如软件中的音量、亮度、对比度等参数调节。
- 数值筛选：如数据列表中的价格范围、时间范围筛选。
- 阈值设置：如报警阈值、权限等级等需要精准数值的场景。

2. 关键注意事项

- 数值约束：`value` 必须在 `min` 和 `max` 之间，超出时会自动被约束到边界值。
- 步长逻辑：`step=0` 表示连续数值（无固定步长），适合需要精准选择的场景；`step>0` 表示离散数值，适合固定增量选择。
- 事件触发：默认 `on_change` 仅在滑块释放时触发，若需实时监听，需订阅 `update:model-value` 事件。
- 节流配置：高频交互场景（如音量调节）建议设置 `throttle`（如 0.1 秒），避免回调触发过于频繁。
- 版本兼容性：`html_id` 属性仅在 2.16.0+ 版本支持，`bind_*` 方法的 `strict` 参数在 3.0.0+ 版本支持。

六、进阶示例：带实时效果的音量调节

结合样式联动，实现拖动滑块时实时改变元素样式，模拟音量调节效果：

```
from nicegui import ui

# 音量滑块（0-100，初始50）
volume_slider = ui.slider(min=0, max=100, value=50, on_change=lambda e: ui.notify(f'音量设置为 {e.value}%'))

# 模拟音量图标（根据数值变化透明度）
volume_icon = ui.icon('volume_up', size='2x1').classes('mt-2')

# 绑定滑块数值到图标透明度（0=完全透明，100=不透明）
volume_icon.bind_style(
    'opacity',
    volume_slider, 'value',
    lambda x: x / 100
)

ui.run()
```

效果：拖动滑块时，音量图标透明度随数值实时变化（数值越小越透明），释放滑块时触发音量设置通知，直观反馈调节效果。